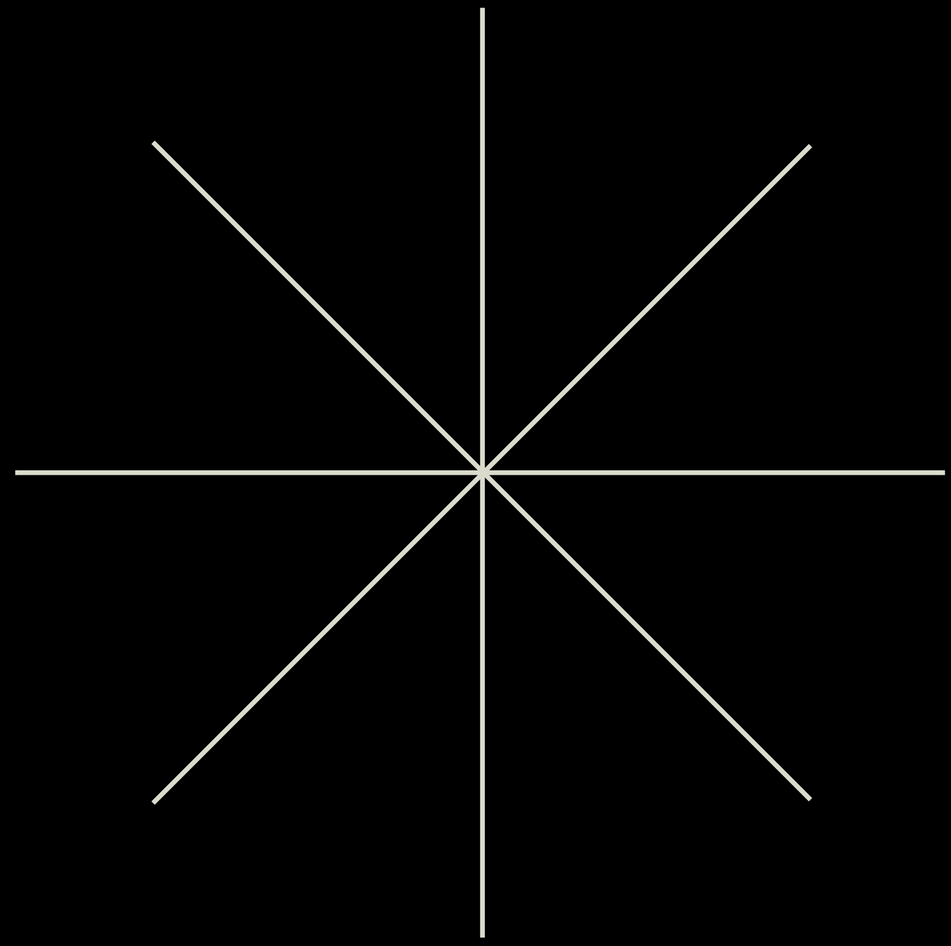
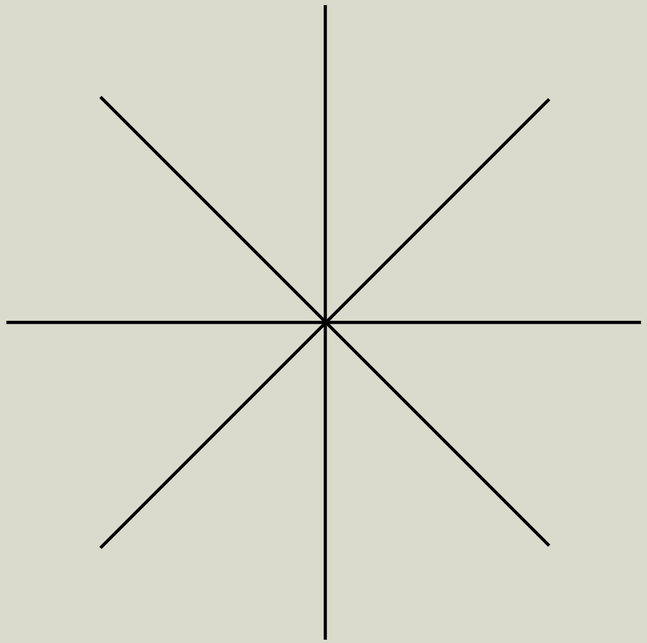
GANG RUI



CONTEXT ENGINEERING

3 strategies that teams like Anthropic, Manus and Cognition use to build AI agents that doesn’t suck



CONTEXT CRISIS

Why your agents fail & what’s at stake

REALITY CHECK

Your agent goes off track at 50k tokens

ANTHROPIC

Handles 500k tokens with ease

MANUS

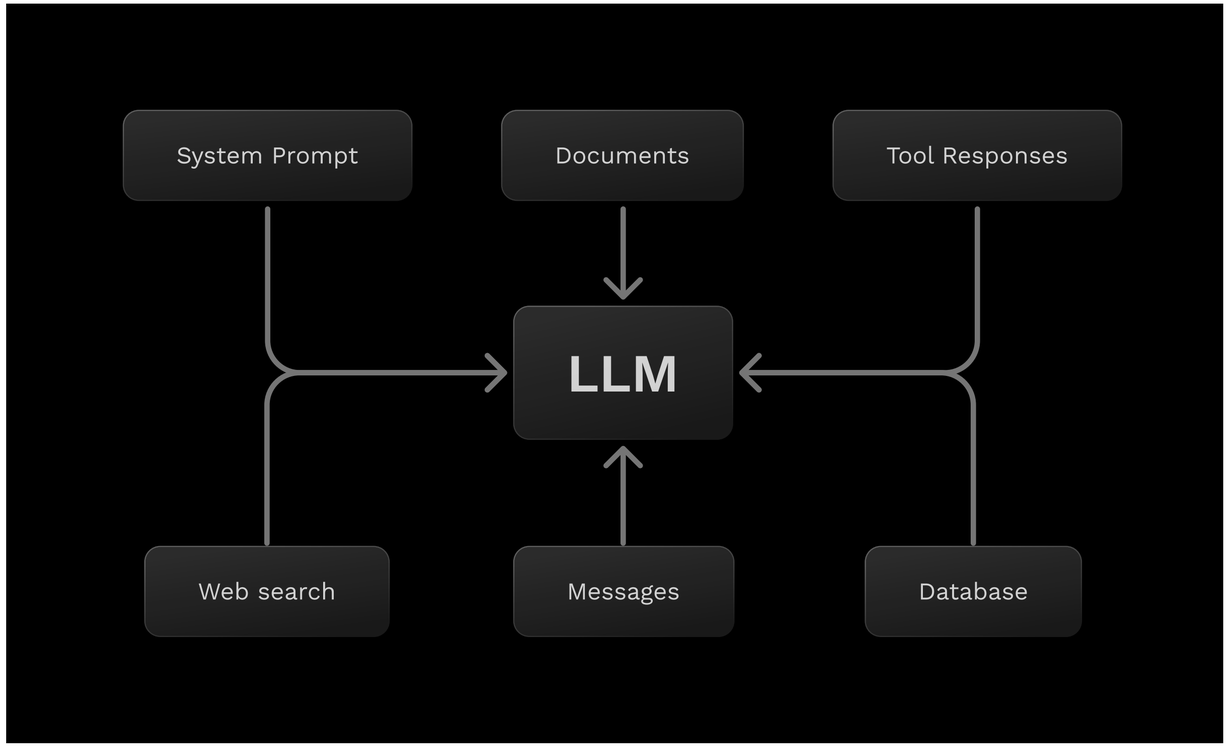
Processes typical tasks with ~50 tool calls without breaking

PARADIGM SHIFT

Prompt engineering → Context Engineering

People associate **prompts with short task descriptions** you'd give an LLM in your day-to-day use. When in every industrial-strength LLM app, **context engineering** is the **delicate art and science of filling the context window with just the right information for the next step**.

~ Andrej Karparthy



CONTEXT CRISIS

4 horsemen of context failure

POISIONING

Error is repeatedly referenced

DISTRACTION

Model over-focuses on long context

CONFUSION

Irrelevant information derails the agent

CLASH

New info conflicts with existing context

SHOCKING TRUTH

67.6% OF CONTEXT = TOOL RESPONSES

WHAT’S AT STAKE

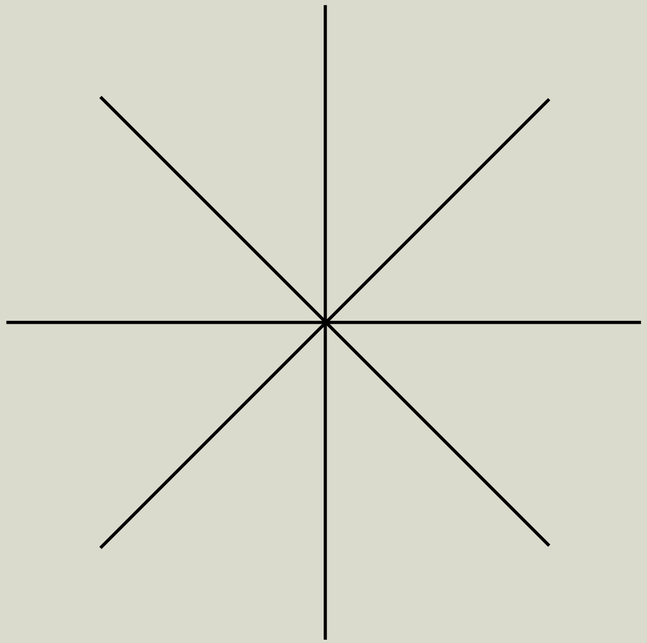
The gap between demo and production

TEAMS STUCK IN DEMO

- Agents crash at 50K tokens
- Can't handle complex workflows
- Fail on real-world data

TEAMS PROFICIENT IN CONTEXT ENGINEERING

- 90% faster research (Anthropic)
- 6 PRs shipped daily with AI (HumanLayer)
- Agents with ability to handle long running tasks (50+ tool calls)



01 SMART RETRIEVAL

From simple search to faceted intelligence

RETRIEVAL REVOLUTION

Battle 1: Simple tools vs RAG techniques

TEAM SIMPLE TOOLS

"Grep and find were sufficient for SWEBench" - Colin, Augment

TEAM RAG

You need hybrid search (keyword + semantic)

POWER OF PERSISTENCE

Why simple tools (sometimes) win

"Agents are dumb but they're persistent... the **agent's persistence is compensating for worse tools**"

~ Colin Flaherty, former founding engineer at Augment

Simple tools **work for code** because

✅ High signal keywords (function names, variables)
✅ Agents retry when first attempt fails
✅ No indexing overhead

But they **break at scale**

❌ "Imagine grep against 10 million files"
❌ "Don't work well with natural language"

CHUNKS TO CONTEXT

Jason Liu’s 4-levels Framwork

LVL 1: MINIMAL CHUNKS

Just text, no metadata

LVL 2: CHUNKS + SOURCE

Enable citations and document loading

LVL 3: MULTI-MODEL

Tables, images, structured data

LVL 4: FACETED SEARCH

Peripheral vision of the data landscape

LVL 4: FACETED SEARCH

Example: Contract Analysis

1. Search “liability provisions” returns **3 contracts (all Signed)**
2. Facets (metadata summary):
   - Signature Status: Signed (45), Unsigned (12), Partially Signed (3)
   - Project: Alpha (23), Beta (18), General Services (19)

The Agent Sees:

- All top results are Signed, but there are 12 Unsigned contracts lurking in the data.
  → Next: zero in on those Unsigned contracts before they’re signed.

Key Insight: “Facets give agents peripheral vision”

When an agent searches `search("liability provisions")`, it gets:

```
<ToolResponse>
  <results query="liability provisions">
    <contract id="MSA-2024-ACME" signature_status="Signed" project="Project Alpha">
      <title>Master Service Agreement - ACME Corp</title>
      <content>Limitation of Liability. In no event shall either party's aggregate liability exc
    </contract>
    <contract id="SOW-2024-BETA" signature_status="Signed" project="Project Beta">
      <title>Statement of Work - Beta Industries</title>
      <content>Liability Cap. Provider's liability is limited to direct damages only, not to exc
    </contract>
    <contract id="AMEND-2024-GAMMA" signature_status="Signed" project="General Services">
      <title>Amendment to Services Agreement - Gamma LLC</title>
      <content>Modified Liability Terms. Section 8.3 is hereby amended to include joint liabilit
    </contract>
  </results>

  <facets>
    <signature_status_facet>
      <value name="Signed" count="45" />
      <value name="Unsigned" count="12" />
      <value name="Partially Signed" count="3" />
    </signature_status_facet>
    <project_facet>
      <value name="Project Alpha" count="23" />
      <value name="Project Beta" count="18" />
      <value name="General Services" count="19" />
    </project_facet>
  </facets>

  <system-instruction>
    Facets expose the complete metadata landscape, revealing information patterns beyond similar

    Key insight: When returned results show bias (e.g., all signed contracts), facets often reve

    Decision framework:
    - Results show bias: investigate facet values not represented in top-k results
    - High clustering in facets: focused filtering more effective than broad search
    - Clear relevance patterns: apply filters autonomously for targeted investigation

    Use signature_status, project, document_type filters strategically based on facet distributi
  </system-instruction>
</ToolResponse>
```

VERDICT ON RETRIEVAL

When to use what

SIMPLE TOOLS

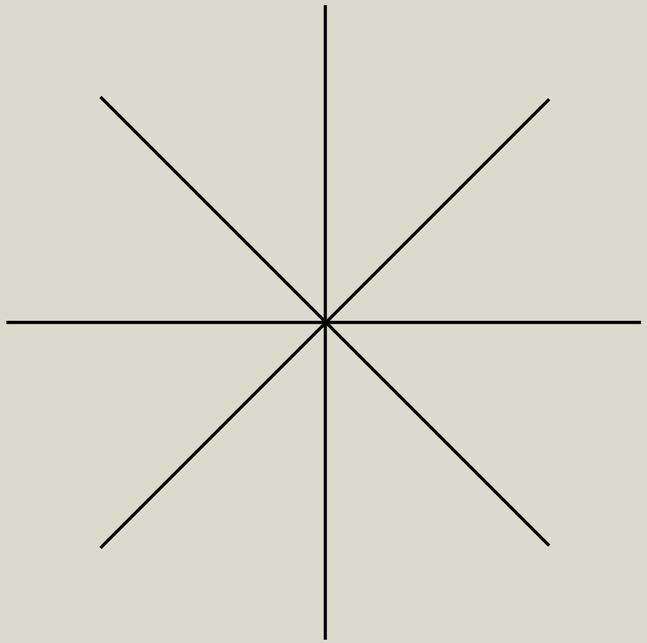
- Code repositories
- High-signal keywords
- Small to medium scale
- Clear file structures

SMART RAG + FACETS

- Documents and natural language
- Large scale (millions of files)
- Complex relationships
- Need for semantic understanding

HYBRID SOLUTION

Agentic loops with simple tools + smart RAG as tools to invoke depending on the use case



02 ISOLATION

When to stay split vs stay united

ARCHITECTURE WARS

Battle 2: One genius vs many specialists

TEAM SINGLE AGENT

Cognition's Warning

"Multi-agents only result in fragile systems"

TEAM MULTI-AGENT

Anthropic's Success Story

- 90.2% performance improvement
- Lead agent + 3-5 parallel subagents
- Cut research time by up to 90%

THE FLAPPY BIRD DISASTER

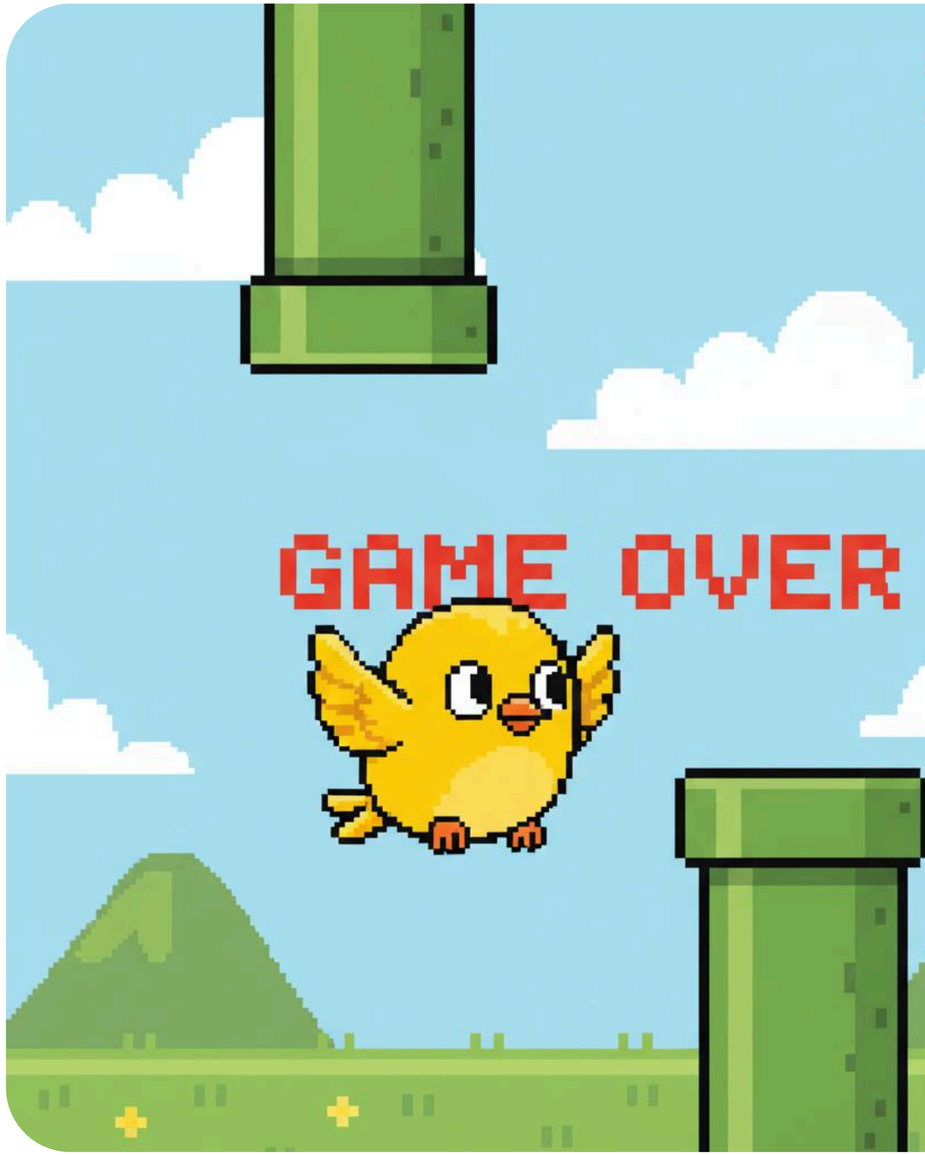
When multi-agent goes wrong

Cognition’s Example:

Task: "Build a Flappy Bird clone"
Subagent 1: "Build game background" → Creates Super Mario Bros background 🍄
Subagent 2: "Build a bird" → Creates realistic bird, wrong physics 🦅
Final Agent: "Combine these" → Disaster! Incompatible pieces 💥

The Problem:

Actions carry implicit decisions, and conflicting decisions carry bad results



CRITICAL DISTINCTION

Read vs write operations

READS CAN BE PARALLEL

- Multiple agents reading files simultaneously
- Running git operations in parallel
- Parsing logs concurrently
- No conflicts possible

WRITES MUST BE SEQUENTIAL

- Multiple agents editing = merge conflicts
- Broken state and race conditions
- Claude Code keeps writes in main thread

WHEN MULTI-AGENT WORKS

Anthropic’s success patterns

Perfect for Multi-Agent:

- "Identify all board members of S&P 500 IT companies"
  - 500 independent searches
  - No coordination needed
- "Find recent news about 50 companies"
  - Parallel web searches
  - Each agent owns its company

The Key: Complete Isolation

- Separate data sources
- No shared state
- Read-only operations

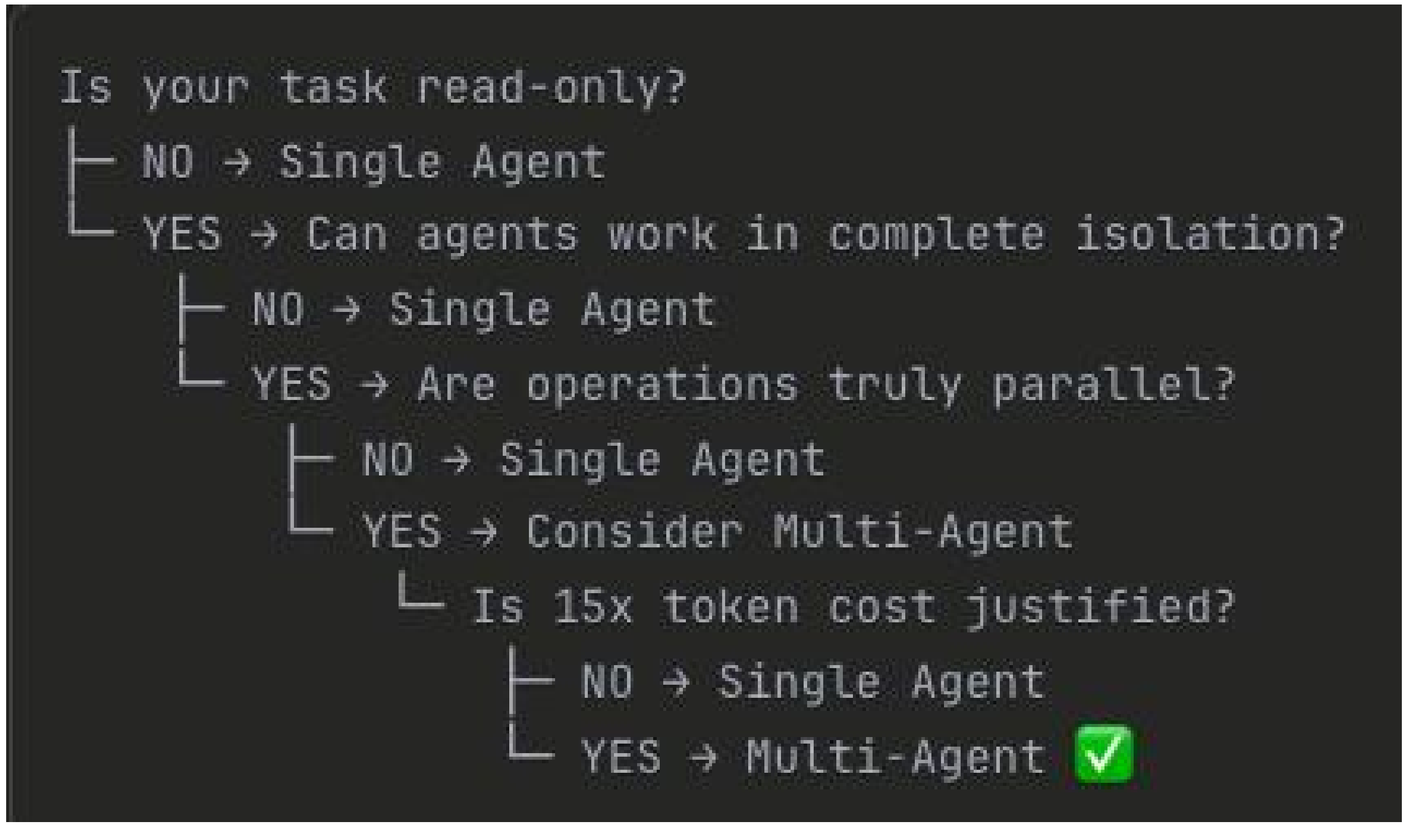
Engineering at Anthropic

How we built our multi-agent research system

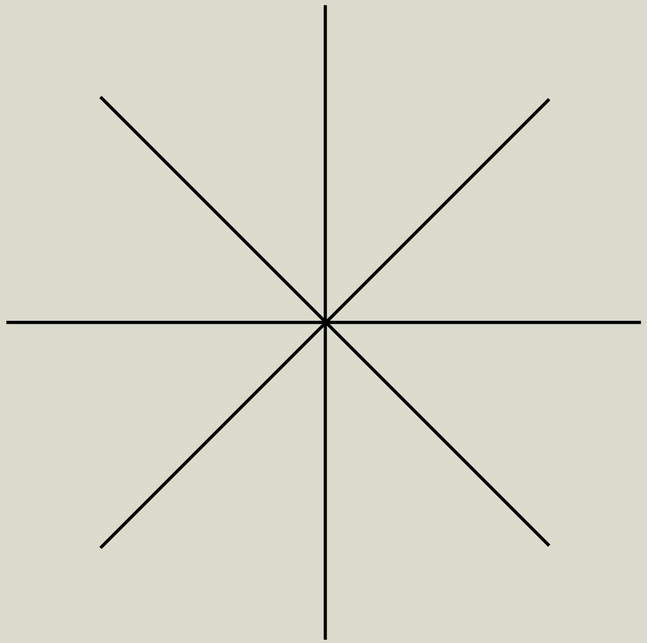
Our Research feature uses multiple Claude agents to explore complex topics more effectively. We share the engineering challenges and the lessons we learned from building this system.

ARCHITECTURE

Decision Framework



Note: Multi-agents uses 15x more tokens that a single chat



03 OFFLOADING

When to stay split vs stay united

CONTEXT OFFLOADING

Your unlimited memory solution

The Breakthrough

"File systems as the ultimate context" - Manus

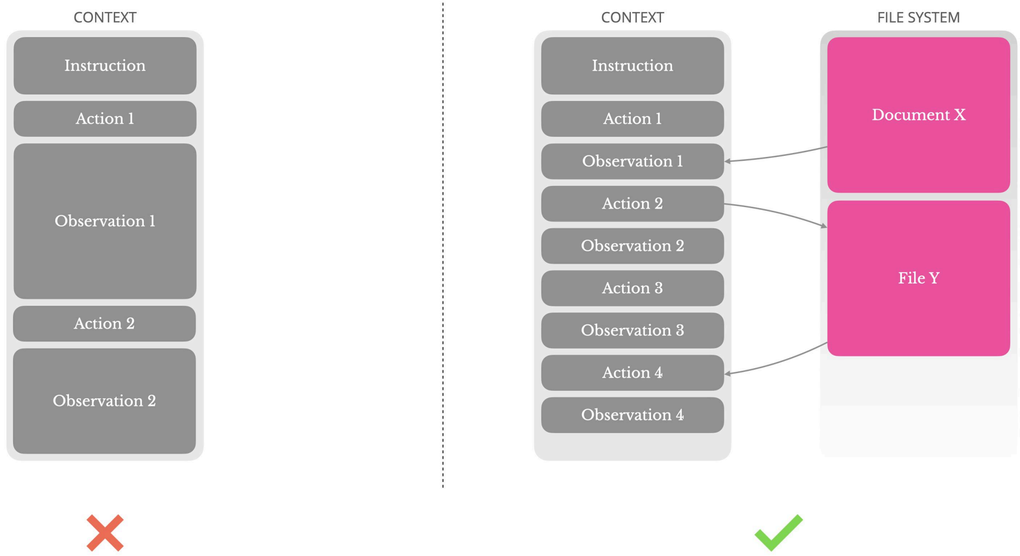
Three Superpowers:

1. Unlimited in size - No context limits
2. Persistent by nature - Survives iterations
3. Directly operable - Read/write on demand

The Result:

"Using file system as structured, externalized memory

Use the File System as Context

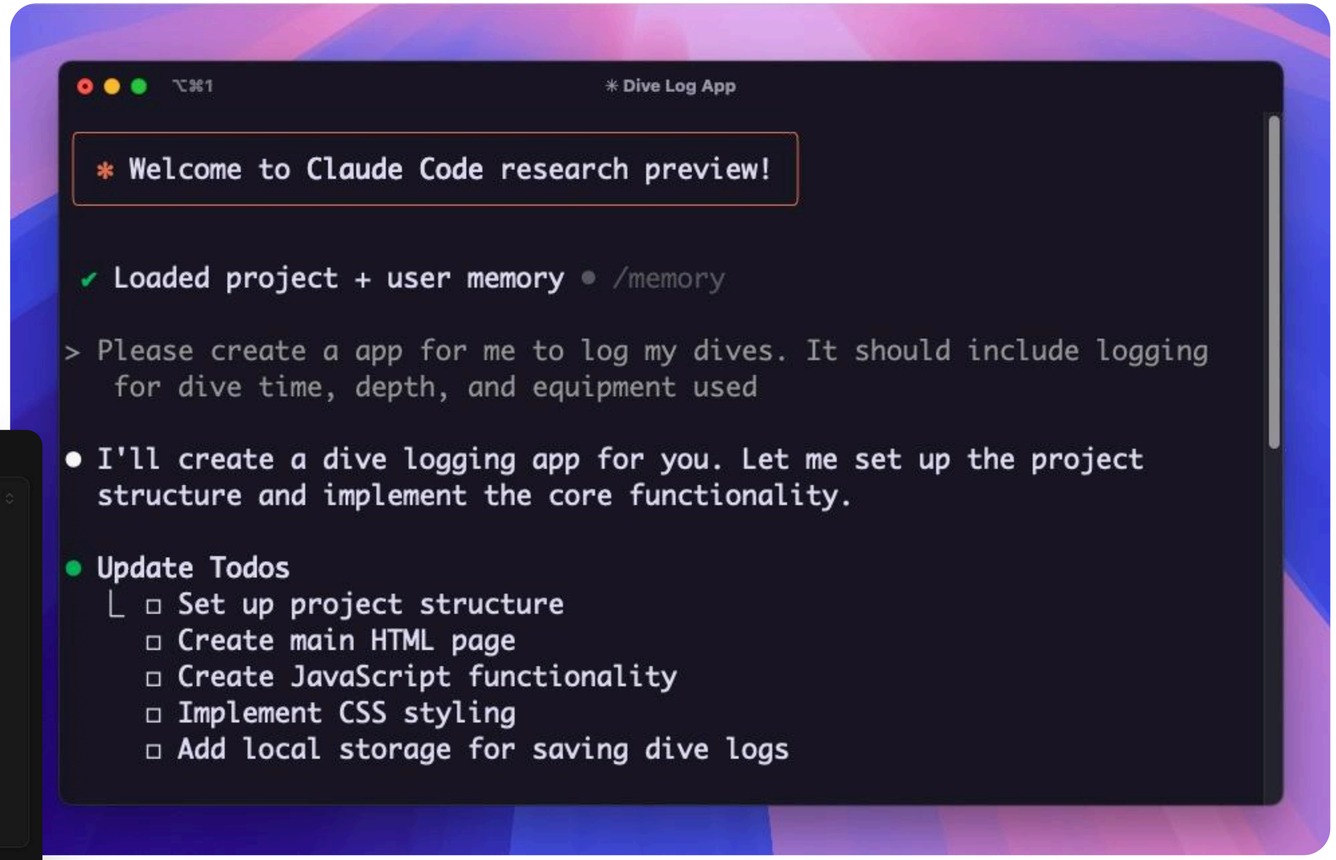


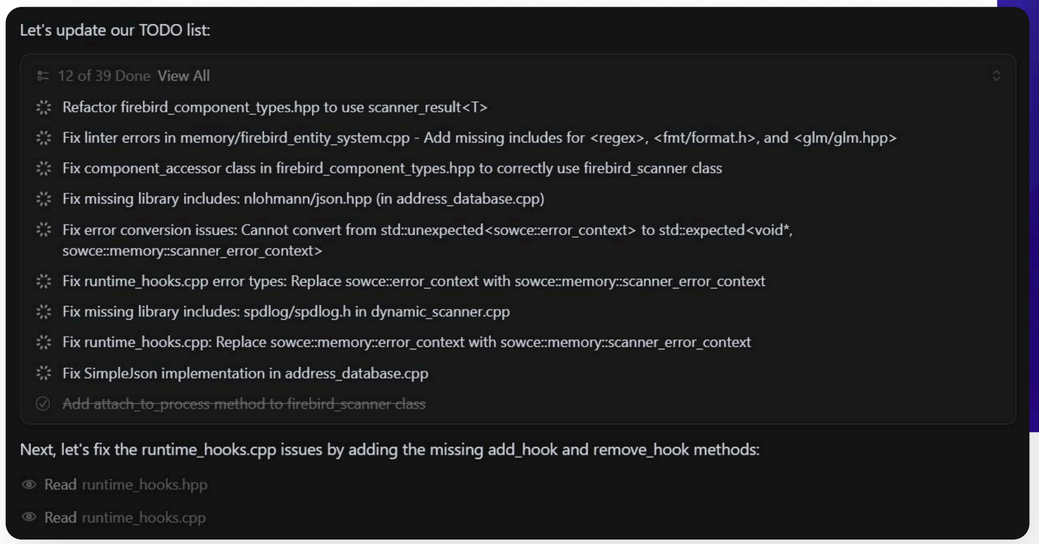
MAGIC OF TODO-LIST

Solving “lost in the middle”

Why It Works:

- Recitation fights attention drift
- "Pushes objectives into recent attention span"
- Typical task: ~50 tool calls - easy to lose focus





WHAT TO OFFLOAD

Smart offloading strategy

Always Offload:

- 📄 Web page content → Keep URL only
- 📚 Document contents → Keep file path
- 🔧 Tool outputs → Keep summary + reference
- 📋 Research plans → External file

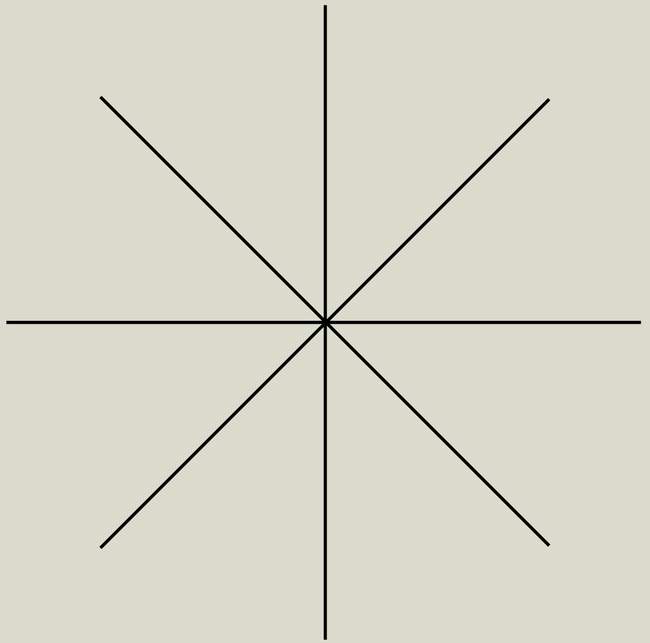
Anthropic’s Limit

“Restrict tool responses to 25,000 tokens by default"

The Principle:

"Compression always designed to be restorable"

- Drop content, keep references
- Agent can retrieve when needed



ACTION PLAN

From theory to production

PLAYBOOK

Context Engineering Checklist

QUICK WINS

Tool Responses Audit

- Calculate what % of context your tools consume (target: <30%)
- Look for token-heavy outputs to offload
- Find metadata you can strip

Enable Planning Capability

- Enable your agents to create plans in external systems
- Update progress throughout execution
- Use recitation to combat "lost-in-the-middle"

Implement Prompt Caching

- Keep context append only
- Note OpenAI & Gemini is automatic while Anthropic is manual
- > 2x cost reduction

ARCHITECTURE

Choose your Retrieval Strategy

- Code repos: Start with grep/find
- Documents: Implement faceted search with metadata
- Test simple before adding embedding-based

Multi-Agent Decision Framework

- Use multi ONLY when: Read-only, isolated context, truly parallel
- Stay single for: Writes, coordination, shared state

Optimise Tool Responses

- Prune metadata aggressively
- XML structures + system instructions in your tool responses to influence agents’ subsequent searches.
- Return facets to reveal patterns
- Many small searches > one broad search

BOTTOM LINE

Every token is a system memory

The Reality

- 90% of agents never leave demo stage
- Winners ship 6 PRs daily with AI
- The difference: Context Engineering

Start with the tool Audit

67.6% of your context awaits optimization